

Transactions — Fundamentals & Execution

Core Concepts, Operation Flow & Memory Architecture

1. What is a Transaction?

A **transaction** is a single logical unit of work consisting of a set of database operations. It generally involves reading and/or updating data within a database.

Example: Transferring ₹500 from Account A to Account B is treated as *one single logical task*, even though it requires multiple low-level database operations to execute.

2. Real-Life Example: ATM Withdrawal

When you withdraw money from an ATM, a series of sequential steps take place:

User Interaction Steps

1. Insert ATM card
2. Read card/account info
3. Authenticate user
4. Select account type

Transaction Execution Steps

5. Enter withdrawal amount
6. Enter PIN
7. Dispense money
8. Complete transaction

★ **Golden Rule: Many operations + One logical task = Transaction**

3. Key Database Operations

At the database level, transactions primarily revolve around two basic operations:

Operation	Database Action	Description
READ	Fetch / Access	Loads data from disk into volatile memory (RAM) for processing.
WRITE	Change / Update	Modifies data in volatile memory (RAM) before permanent storage.

4. Comprehensive Example: Transfer ₹500 (A → B)

Initial State: Account A = ₹1000 | Account B = ₹2000

Step	Command	Memory State	Action Description
Step 1	READ(A)	A = 1000	Fetches Account A balance into RAM.
Step 2	A = A - 500	A = 500	CPU modifies balance in RAM.
Step 3	WRITE(A)	A = 500	Writes modified balance to RAM workspace.

5. The COMMIT & ROLLBACK Operations ★

Intermediate changes made during a transaction reside in volatile memory (RAM). Permanent persistence requires explicit termination controls:

COMMIT

Permanently saves all modifications made by the transaction from RAM to the actual Database/Storage.

ROLLBACK

Undoes/reverts all uncommitted changes, returning the database to its initial consistent state if an error occurs.

Account State Summary:

Account	Before Transaction	After COMMIT
Account A	₹1000	₹500
Account B	₹2000	₹2500

6. Basic Transaction Structure & Memory Flow

```
-- Transaction Structure for Account Transfer READ(A); A := A - 500; WRITE(A); READ(B); B := B + 500; COMMIT;
```

7. Role of RAM in Transaction Processing

CPUs cannot efficiently perform mathematical operations directly on slow physical disk storage. Therefore, data travels through a layered system architecture:



8. Exam-Oriented Summary & Memory Tricks

🧠 Super-Easy Memory Trick

R - W - C - R

- Read = Take / Fetch data
- Write = Change / Modify data
- Commit = Save permanently
- Rollback = Undo changes

🔥 One-Line Revision

A **transaction** is a collection of operations forming a single logical unit of work, primarily using READ and WRITE, finalized with COMMIT to save changes permanently.

Next Topics: Transaction States (Active, Partially Committed, Committed, Failed, Aborted) and **ACID Properties** (Atomicity, Consistency, Isolation, Durability).